

Functions (module 3)

Parameterized functions :

```
class Program{  
    void sum(int a, int b){  
        int s=a+b;  
        System.out.println(s);  
    }  
    void product(int a, int b){  
        int s=a*b;  
    }  
    public static void main(String args[]){  
        Program obj=new Program();  
  
        obj.sum(5,10); // prints 15  
        obj.product(2,3); // prints 6  
    }  
}
```

Formal Parameters

Actual Parameters

```
Scanner sc=new Scanner(System.in);  
int a=sc.nextInt();  
int b=sc.nextInt();
```

```
obj.sum(a,b); // prints value of a+b (a and b are user given inputs)
```

```
obj.product(a,b); // prints value of a*b (a and b are user given inputs)  
}  
}
```

Call by value Example:

```
class Program{  
    void sum(int a, int b){  
        int s=a+b;  
        System.out.println(s);  
    }  
    public static void main(String args[]){  
        Program obj=new Program();  
        Scanner sc=new Scanner(System.in);  
        obj.sum(5, 10);  
        int a=sc.nextInt();  
        int b=sc.nextInt();  
        obj.sum(a,b);  
    }  
}
```

Pure function

Call by value / Pass by value

Call by reference:

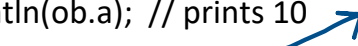
```
class Program{  
    int a;  
    void change( Program obj ){  
        obj.a=50;  
    }  
}
```

Impure Function



```
public static void main(String args[]){  
    Program ob=new Program();  
    ob.a=10;  
    System.out.println(ob.a); // prints 10  
    ob.change(ob);  
    System.out.println(ob.a); // prints 50  
}
```

Call by reference / Pass by reference



Method signature:

In Java, a **method signature** refers to the combination of the **method name** and its **parameter list** (types and order of parameters). The method signature is used to uniquely identify a method within a class.

Examples

Method header

```
public void print(int a)  
public void print(double b)  
public void print(int a, int b)
```

Method Signature

```
print(int)  
print(double)  
print(int, int)
```

Function Overloading (Method Overloading) in Java

Function Overloading (also called **Method Overloading** in Java) is a feature that allows multiple methods in the same class to have **the same name but different parameter lists**. It's compile time polymorphism.

Rules for Function Overloading

To overload a method, keeping the function name same, you must change at least one of the following:

1. **Number of parameters**
2. **Type of parameters**
3. **Order of parameters**

Note: The return type is NOT considered for overloading.

Example:

```
class Apple{
```

```
void sum(int a){  
System.out.println(a+5);  
}
```

```
void sum(int a, int b){  
System.out.println(a+b);  
}
```

```
void sum(int a, float b){  
System.out.println(a+b);  
}
```

```
void sum(float b, int a){  
System.out.println(a+b);  
}
```

```
public static void main(String args[]){  
Apple obj=new Apple();
```

```
obj.sum(10);           // prints 15
```

```
obj.sum(10,15);        // prints 25
```

```
obj.sum(15, 12.5);     // 27.5
```

```
obj.sum(10.5, 10);     // 20.5
```

```
}  
}
```

State and behaviour:

The **state** of an object refers to its **properties (attributes or fields)** and their current values.

- It is represented by **instance variables** (data members) of a class.
- The state can change over time as the object undergoes modifications.

The **behaviour** of an object defines **what the object can do**.

- It is represented by **methods** in a class.
- Methods operate on the object's state and can change it.

```
void sum(int a, int b){ -----→ function header/ prototype
```

```
// function body
```

```
}
```

Example Short Question:

Write prototype/header of a function that takes a String and returns an integer

Answer: `int function(String s)`

Write prototype/header of a function “hello” that takes an integer and returns a character

Answer: `char hello(int a)`

Formal Parameters

Formal parameters are **the parameters defined in the method declaration**. They act as placeholders for the values that will be passed when calling the method.

Example:

```
void show(int a) { // "a" is a formal parameter
```

```
    System.out.println("Hello " + name);
```

```
}
```

Actual Parameters

Actual parameters are **the values/arguments passed to a method during a method call**. These values replace the formal parameters.

```
obj.show(10); // 10 is an actual parameter
```

```
int k=sc.nextInt();
```

```
obj.show(k); // k is an actual parameter
```

Call by Value

In **Call by Value**, a copy of the actual parameter's value is passed to the method.

Changes made inside the method **do not affect the original value** in the calling function. It means changes in formal parameters do not reflect on actual parameters.

Call by Reference

In **Call by Reference**, a reference (memory address) of the actual parameter is passed, so changes inside the method **affect the original object**.

****Java does NOT support call by reference for primitive types but supports it for objects (since objects are references).**

Pure Functions

Pure functions only depend on their input parameters and local members and **do not modify the state of the object or have side effects**

Impure Functions

Impure functions can have unpredictable behaviour and **can modify the state of the object or have side effects.**

Parameterized Returning function with Nested Call

```
class Program{  
  
    int square(int x){  
  
        return x*x;  
  
    }  
  
    public static void main(String args[]){  
  
        Program obj=new Program();  
  
        System.out.println(obj.square(2));           //prints 4  
  
        System.out.println(obj.square(obj.square(2))); // prints 16  
  
    }  
  
}
```

Function with multiple return statements (However, it returns only one value based on condition)

Example: a function that returns 1 if a number is even otherwise it returns 0

```
int check(int n){  
    if(n%2==0) return 1;  
    else return 0;  
}
```

**** Observe that a function can contain multiple return statements, but ultimately, it returns a single value.**

20. Define a class Equation to print the result of the following equation:

$$\frac{n!}{r!(n-r)!}$$

member functions:

int fact(int n) to return factorial of n

void printResults(int r, int n) to print the result of the above function using int fact()

Write the main function to call

```
import java.util.Scanner;
class Equation{

int fact(int n){
    int f=1;
    for(int i=1; i<=n; i++){
        f*= i;
    }
    return f;
}

void printResults(int r, int n){

double s= fact(n)/( fact(r) * fact(n-r) );
System.out.println(s);
}

public static void main(String args[]){
    Program obj=new Program();
    Scanner sc=new Scanner(System.in);

    System.out.println("Enter value of n");
    int n=sc.nextInt();

    System.out.println("Enter value of r");
    int r=sc.nextInt();

    obj.printResults(r,n);
}
}
```